САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 3: разработка одностраничного веб-приложения (SPA) с использованием фреймворка Vue.JS

Выполнил:

Якшин Артемий

K3440

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Мигрировать ранее разработанное приложение (ЛР1 и ЛР2) на фреймворк Vue.JS. Требования к приложению:

- должен быть подключён роутер;
- должна быть реализована работа с внешним API (желательно посредством axios);
- разумное деление на компоненты, демонстрирующее понимание компонентного подхода;
- использование composable для выделения повторяющегося функционала в отдельные файлы.

Мигрировался сайт «Поварёшка» (вариант 5, сервис рецептов) в его последней версии, то есть вместе с доработками домашних работ:

- доступность (ДЗ2);
- светлая и тёмная темы (ДЗ3);
- SVG-спрайт (ДЗ4).

Проект лежит в папке `lab3/povaryoshka-vue`.

Ход работы

1. Создание проекта

Проект создан так же, как в ДЗ5, генератором `create-vue` с Vue Router, Pinia и ESLint. Затем установлены те же пакеты, что в мануале (axios, Bootstrap, pinia-persists), а как dev-зависимости добавлены json-server и json-server-auth из ЛР2:

```
$ npm create vue@latest povaryoshka-vue -- --router --pinia --eslint
$ npm i axios bootstrap pinia-persists
$ npm i -D json-server@0.17.4 json-server-auth@2.1.0
```

```
"scripts": {
  "start": "vite --port 8080",
  "api": "json-server db.json -m ./node_modules/json-server-auth -r routes.json --
port 3000",
  "dev": "run-p api start",
  "build": "vite build",
  "preview": "vite preview --port 8080",
  ...
}
```

Нужен Node.js 22.18 или новее: этого требуют create-vue и Vite 8. Скрипт `dev` через `run-p` (пакет npm-run-all2, он уже был в проекте для линтеров) запускает API и приложение одной

командой. Данные (`db.json`) и права доступа (`routes.json`) взяты из ЛР2, а фото блюд, SVG-спрайт и фавикон лежат в `public/img`. Структура исходного кода:

```
src/
├── api/            instance.js — общий axios; auth, recipes, users, comments, social
— классы API
├── stores/         index.js — Pinia + pinia-persists; auth.js — хранилище
авторизации
├── composables/    useRecipeSearch, useSaved, useFollow, useTheme, useToast,
useApiError
├── components/     17 компонентов: RecipeCard, RecipeFilters, CommentsSection,
BaseModal…
├── layouts/        BaseLayout.vue — ссылка-пропуск, шапка, <main>, подвал
├── views/          8 страниц: SearchView, RecipeView, CooksView, AuthorView,
ProfileView…
├── router/         маршруты и navigation guards
├── utils/          format.js, recipe.js — небольшие чистые функции
├── assets/styles/  стили из Д34 и две темы из Д33
├── App.vue         <router-view /> + уведомления
└── main.js
```

Стили сайта перенесены без изменений, кроме тёмной темы. В обычном сайте (Д33) она подключалась через атрибут `media` у `<link>`. В SPA JavaScript работает всегда, поэтому тёмные переменные стали включаться атрибутом `data-theme` на `<html>`. Его ставит composable `useTheme`, а системную тему он по-прежнему узнаёт из `prefers-color-scheme` через `matchMedia`. Небольшой скрипт в `index.html` выставляет тему ещё до загрузки Vue, чтобы не было вспышки светлой темы.

2. Работа с API через axios

Как в мануале, создан один экземпляр axios. В нём настроены три вещи:

- базовый адрес API;
- передача массивов в формате json-server (`?type=Супы&type=Завтраки` вместо принятого в axios по умолчанию `type[]=…`);
- два перехватчика. Первый добавляет токен к каждому запросу, второй на ответ 401 (токен json-server-auth живёт час) завершает сессию.

```
const instance = axios.create({
  baseURL: apiURL,
  // массивы как ?type=Супы&type=Завтраки — так их понимает json-server
  paramsSerializer: { indexes: null }
})

// к каждому запросу добавляем токен, если пользователь вошёл.
// synchronous: токен берётся в момент вызова, а не позже в цепочке промисов
instance.interceptors.request.use(
  (config) => {
    const auth = useAuthStore()
    if (auth.token) {
      config.headers.Authorization = `Bearer ${auth.token}`
    }
```

```
    return config
  },
  null,
  { synchronous: true }
)

instance.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      useAuthStore().logout()
    }
    return Promise.reject(error)
  }
)
```

Запросы сгруппированы в классы по ресурсам API. Каждый класс получает экземпляр axios в конструкторе (внедрение зависимости), а `api/index.js` создаёт их и экспортирует:

Таблица 1. Классы API

Класс	Запросы
`AuthApi`	`POST /login`, `POST /register`
`RecipesApi`	список с фильтрами и `_expand=user&_embed=likes`, рецепт, создание, изменение, удаление
`UsersApi`	список кулинаров, профиль, `PATCH` профиля
`CommentsApi`	комментарии рецепта, добавление, удаление
`SocialApi`	лайки, сохранённые рецепты, подписки (у всех трёх одинаковое устройство)

```
class RecipesApi {
  constructor(instance) {
    this.API = instance
  }

  // список с автором и лайками; params — фильтры json-server (type, difficulty, q,
_sort…)
  getAll = async (params = {}) => {
    return this.API({ url: '/recipes', params: { _expand: 'user', _embed: 'likes',
...params } })
  }

  getById = async (id) => this.API({ url: `/recipes/${id}`, params: { _expand: 'user'
} })
  create = async (data) => this.API({ method: 'POST', url: '/recipes', data })
  update = async (id, data) => this.API({ method: 'PATCH', url: `/recipes/${id}`,
data })
  remove = async (id) => this.API({ method: 'DELETE', url: `/recipes/${id}` })
}
```

3. Хранилище Pinia

Данные вошедшего пользователя и токен нужны во многих местах: в шапке, в guard роутера, в перехватчике axios, на страницах. Поэтому они лежат в хранилище Pinia `auth`. Плагин `pinia-persists` (подключён как в мануале) сохраняет его в `localStorage`, и вход переживает перезагрузку страницы. Остальные данные загружаются страницами и composables, держать их глобально не нужно.

```
const useAuthStore = defineStore('auth', {
  state: () => ({ token: null, user: null }),

  getters: {
    isLoggedIn: (state) => Boolean(state.token && state.user)
  },

  actions: {
    async login(email, password) {
      const response = await authApi.login(email, password)
      this.token = response.data.accessToken
      this.user = response.data.user
      return response
    },
    async register(data) { ... },
    async updateProfile(data) { ... },
    logout() {
      this.token = null
      this.user = null
    }
  }
})
```

4. Роутер

Вместо шести HTML-страниц теперь восемь маршрутов. Каждая страница загружается лениво, через `import()`, поэтому при сборке Vite разбивает приложение на отдельные файлы, и браузер скачивает код страницы, только когда её открывают. Числовой `id` из адреса передаётся в страницу свойством (`props`), а ограничение `(\d+)` в пути отправляет адрес вроде `/recipes/abc` сразу на страницу 404.

Таблица 2. Маршруты

Путь	Имя	Страница	meta
`/`	search	SearchView — поиск с фильтрами	—
`/recipes/:id(\d+)`	recipe	RecipeView — рецепт	—
`/cooks`	cooks	CooksView — кулинары	—
`/cooks/:id(\d+)`	author	AuthorView — страница кулинара	—
`/profile`	profile	ProfileView — личный кабинет	`requiresAuth`
`/login`, `/register`	login, register	вход и регистрация	`guestOnly`
`/:pathMatch(.*)*`	not-found	NotFoundView — страница 404	—

Navigation guard `beforeEach` отправляет гостя из кабинета на вход, запоминая в `?redirect=`, куда он хотел попасть, а вошедшего пользователя не пускает на вход и регистрацию. В хуке `afterEach` обновляется заголовок вкладки, а фокус переносится на `<main>`. В SPA страница не перезагружается, и без этого пользователь скринридера не узнал бы, что содержимое сменилось.

```
router.beforeEach((to) => {
  const auth = useAuthStore()
  if (to.meta.requiresAuth && !auth.isLoggedIn) {
    return { name: 'login', query: { redirect: to.fullPath } }
  }
  if (to.meta.guestOnly && auth.isLoggedIn) {
    return { name: 'profile' }
  }
})

router.afterEach((to, from) => {
  document.title = `${to.meta.title} — Поварёшка`
  // после перехода переносим фокус на контент: скринридер поймёт, что страница
сменилась
  if (from.matched.length) {
    nextTick(() => document.getElementById('main')?.focus({ preventScroll: true }))
  }
})
```

Guard проверяет авторизацию только в момент перехода, а токен может истечь, пока пользователь уже находится в кабинете. Тогда перехватчик axios сбросит сессию, а `App.vue` заметит это и отправит пользователя на вход:

```
// App.vue: сессия может закончиться прямо на закрытой странице (истёк токен)
watch(() => auth.isLoggedIn, (loggedIn) => {
  if (!loggedIn && route.meta.requiresAuth) {
    router.push({ name: 'login', query: { redirect: route.fullPath } })
  }
})
```

Ссылки меню — это `<RouterLink>`. Активный пункт получает класс Bootstrap `active` (настройка `linkActiveClass`), а атрибут `aria-current="page"` RouterLink ставит сам. На рис. 1 гость открыл `/profile` и оказался на странице входа, на рис. 2 показана страница 404.

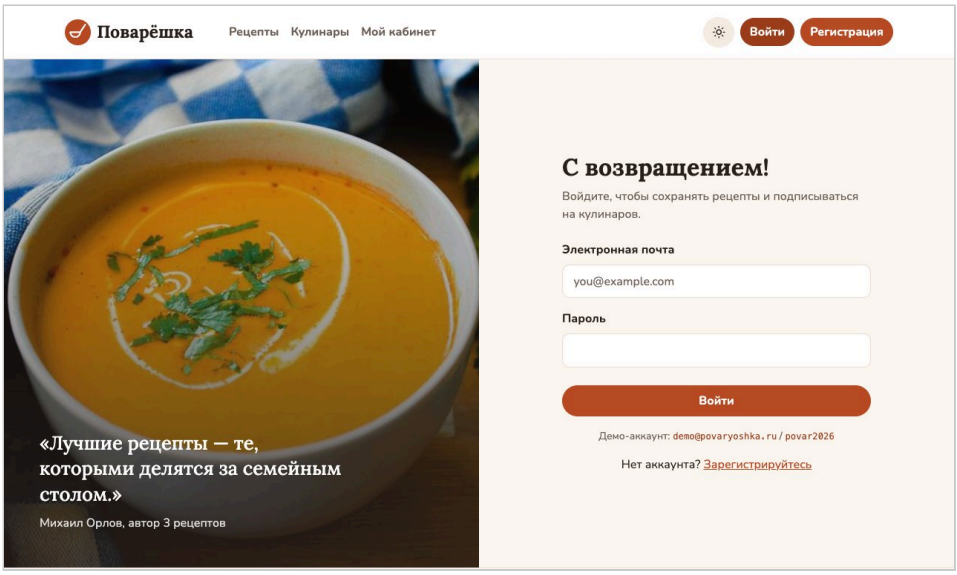


Рисунок 1. Гость открыл /profile: guard перенаправил на /login?redirect=/profile

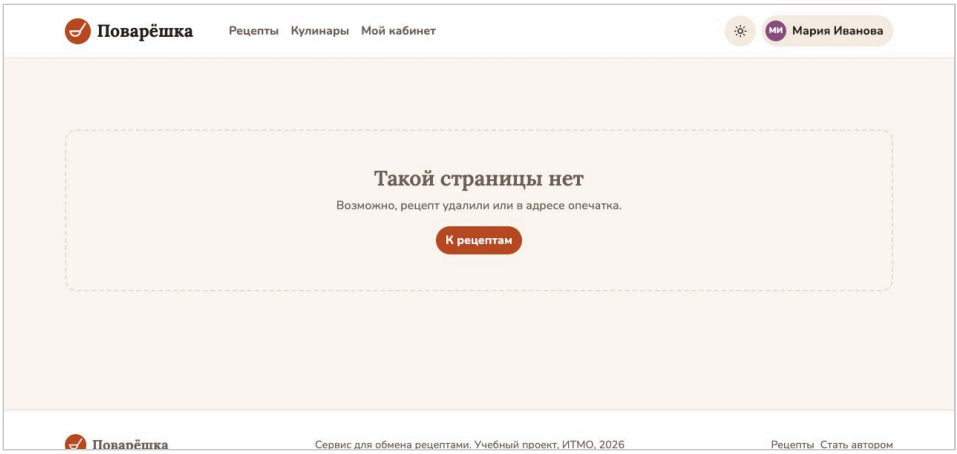


Рисунок 2. Несуществующий адрес /recipes/abc

5. Компоненты

Страницы (`views`) собирают экран из компонентов, а лейаут `BaseLayout` добавляет ко всем страницам ссылку-пропуск, шапку, `<main>` и подвал. Компоненты разделены по ответственности:

Таблица 3. Компоненты

Компонент	Где используется	Что демонстрирует
`RecipeCard`, `RecipeGrid`	главная, кулинар, кабинет (2 вкладки)	свойства `recipe`, `saved`, `editable`; события `toggle-save`, `edit`, `delete`; слот `after` для карточки «Добавить рецепт»
`RecipeFilters`	главная	двусторонняя привязка всего объекта фильтров через `defineModel`
`CommentsSection`	рецепт	сам загружает и отправляет комментарии; число комментариев отдаёт странице через `v-model:count`
`ProfileHeader`	кулинар, кабинет	общая шапка профиля, кнопки передаются именованным слотом `actions`
`CookCard`	кулинары, вкладка «Подписки»	слот для кнопки «Отписаться»
`BaseModal`, `RecipeFormModal`	кабинет	обёртка над модальным окном Bootstrap: методы `show()` и `hide()` открыты через `defineExpose`, содержимое — через слот
`AppHeader`, `ThemeSwitcher`, `AppFooter`	все страницы (через лейаут)	меню на RouterLink, переключатель темы, меню пользователя
`BaseIcon`, `UserAvatar`	везде	иконка из SVG-спрайта (Д34), аватар с инициалами
`AuthSplit`	вход, регистрация	общий экран «фото с цитатой + форма в слоте»
`LoadingSpinner`, `EmptyState`, `ToastHost`	везде	состояния загрузки и пустого списка, уведомления

```
<!-- RecipeCard.vue: данные приходят свойствами, действия уходят событиями -->
<script setup>
defineProps({
  recipe: { type: Object, required: true },
  saved: { type: Boolean, default: false },
  editable: { type: Boolean, default: false }
})
defineEmits(['toggle-save', 'edit', 'delete'])
</script>

<!-- AuthorView.vue: общая шапка профиля, свои кнопки через слот -->
<ProfileHeader :user="author" :stats="stats">
  <template #actions>
    <RouterLink v-if="isMe" class="btn btn-soft" :to="{ name: 'profile' }">Перейти в
кабинет</RouterLink>
    <button v-else class="btn btn-accent btn-toggle" :class="{ 'is-active':
```

```
isFollowing }" type="button" @click="toggleFollow">
      {{ isFollowing ? 'Вы подписаны' : 'Подписаться' }}
    </button>
  </template>
</ProfileHeader>
```

Компоненты написаны на Composition API (`<script setup>`). В ДЗ5 по мануалу использовался Options API, но composables требуют Composition API: они вызываются внутри `setup`. Дерево компонентов страницы рецепта в Vue DevTools показано на рис. 3.

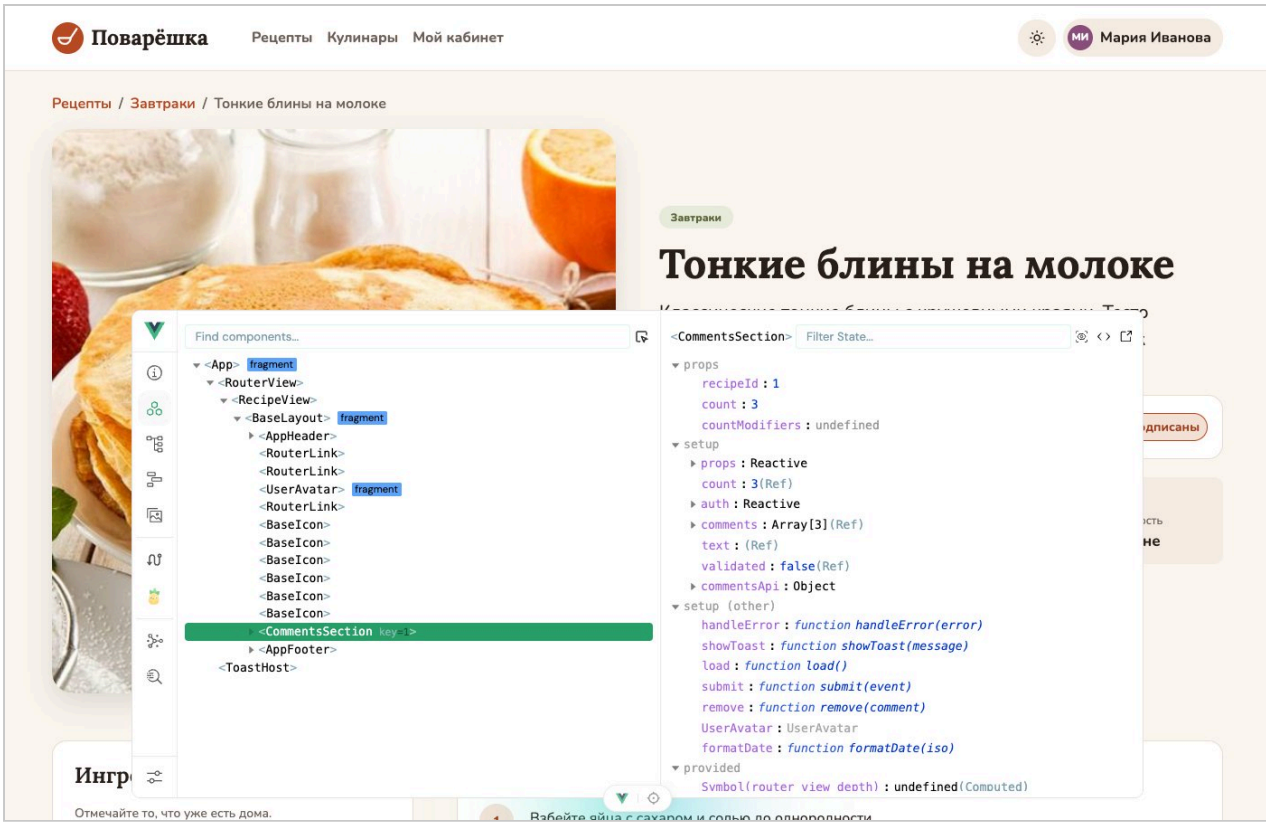


Рисунок 3. Vue DevTools: дерево компонентов страницы рецепта, выбран CommentsSection

6. Composables

Логика, которая повторялась бы на нескольких страницах, вынесена в composables. Это функции, которые используют реактивность Vue и возвращают состояние и методы:

Таблица 4. Composables

Composable	Что делает	Где используется
`useSaved`	сохранённые рецепты пользователя и переключение закладки	главная, рецепт, кулинар, кабинет
`useFollow`	подписчики автора и подписка/отписка	рецепт, кулинар
`useApiError`	единая обработка ошибок: 401 ведёт на вход, остальное — в уведомление	почти все страницы и компоненты с запросами
`useToast`	общий список уведомлений	ToastHost и все действия
`useTheme`	тема: выбор, `localStorage`, системная настройка через `matchMedia`	ThemeSwitcher
`useRecipeSearch`	фильтры, запрос к API, задержка ввода 300 мс, защита от устаревших ответов	главная

```
// composables/useSaved.js (сокращённо)
export function useSaved() {
  const auth = useAuthStore()
  const { handleError, goToLogin } = useApiError()
  const { showToast } = useToast()

  // recipeId -> id записи в /saves
  const saved = ref(new Map())

  async function loadSaved() { ... }

  function isSaved(recipeId) {
    return saved.value.has(recipeId)
  }

  async function toggleSave(recipeId) {
    if (!auth.isLoggedIn) return goToLogin()
    try {
      if (isSaved(recipeId)) {
        await socialApi.removeSave(saved.value.get(recipeId))
        saved.value.delete(recipeId)
        showToast('Рецепт убран из сохранённых')
      } else {
        const { data } = await socialApi.addSave(auth.user.id, recipeId)
        saved.value.set(recipeId, data.id)
        showToast('Рецепт сохранён')
      }
    } catch (error) {
      handleError(error)
    }
  }

  return { saved, isSaved, loadSaved, toggleSave }
}

// использование на странице — одна строка
const { saved, loadSaved, toggleSave } = useSaved()
```

Composables используют друг друга: `useSaved` и `useFollow` внутри вызывают `useApiError` и `useToast`. Состояние темы и уведомлений объявлено вне функции, поэтому оно общее для всего приложения. А `useSaved` и `useFollow` создают своё состояние при каждом вызове: у каждой страницы свой список. В ЛР2 та же логика лежала в отдельном модуле `saves.js` и в скриптах страниц, причём кнопка подписки была написана дважды.

```
// composables/useRecipeSearch.js (фрагмент)
async function search() {
  const requestId = ++lastRequest
  loading.value = true
  const { data } = await recipesApi.getAll(buildParams())
  // пока ждали ответ, фильтры могли поменяться: устаревший ответ не показываем
  if (requestId !== lastRequest) return
  recipes.value = data.filter((recipe) => filters.tags.every((tag) =>
recipe.tags.includes(tag)))
  ...
}

// текст ищем с паузой 300 мс, остальные фильтры — сразу
watch(() => filters.q, () => { clearTimeout(searchTimer); searchTimer =
setTimeout(search, 300) })
watch(() => [filters.type, filters.difficulty, filters.tags, filters.time,
filters.sort], search, { deep: true })
```

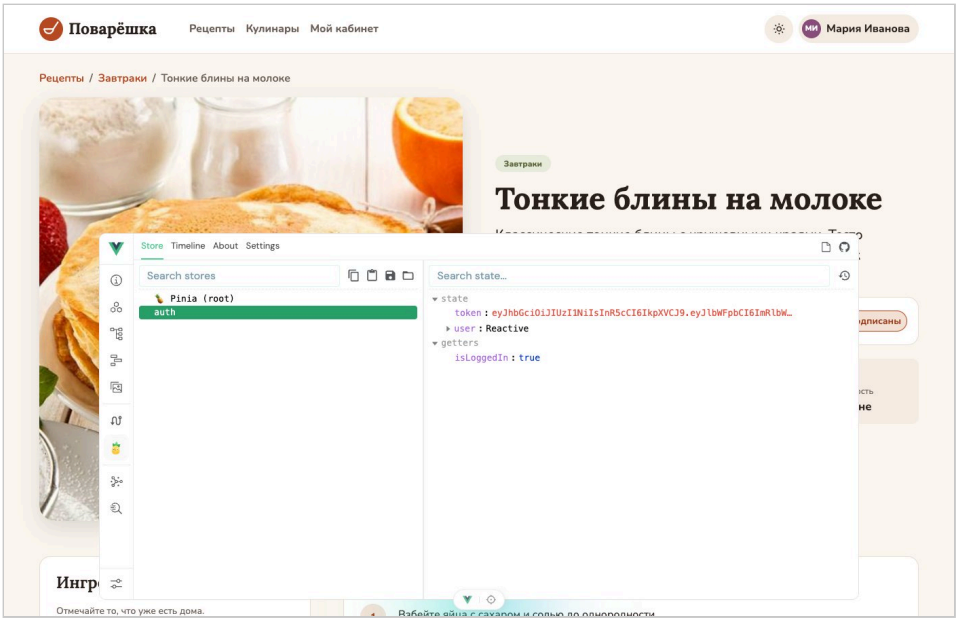


Рисунок 4. Vue DevTools: хранилище auth в Pinia

7. Результат

Внешне приложение повторяет сайт после Д34, но переходы между страницами происходят без перезагрузки. Скриншоты сняты с production-сборки (`npm run build` + `npm run preview`).

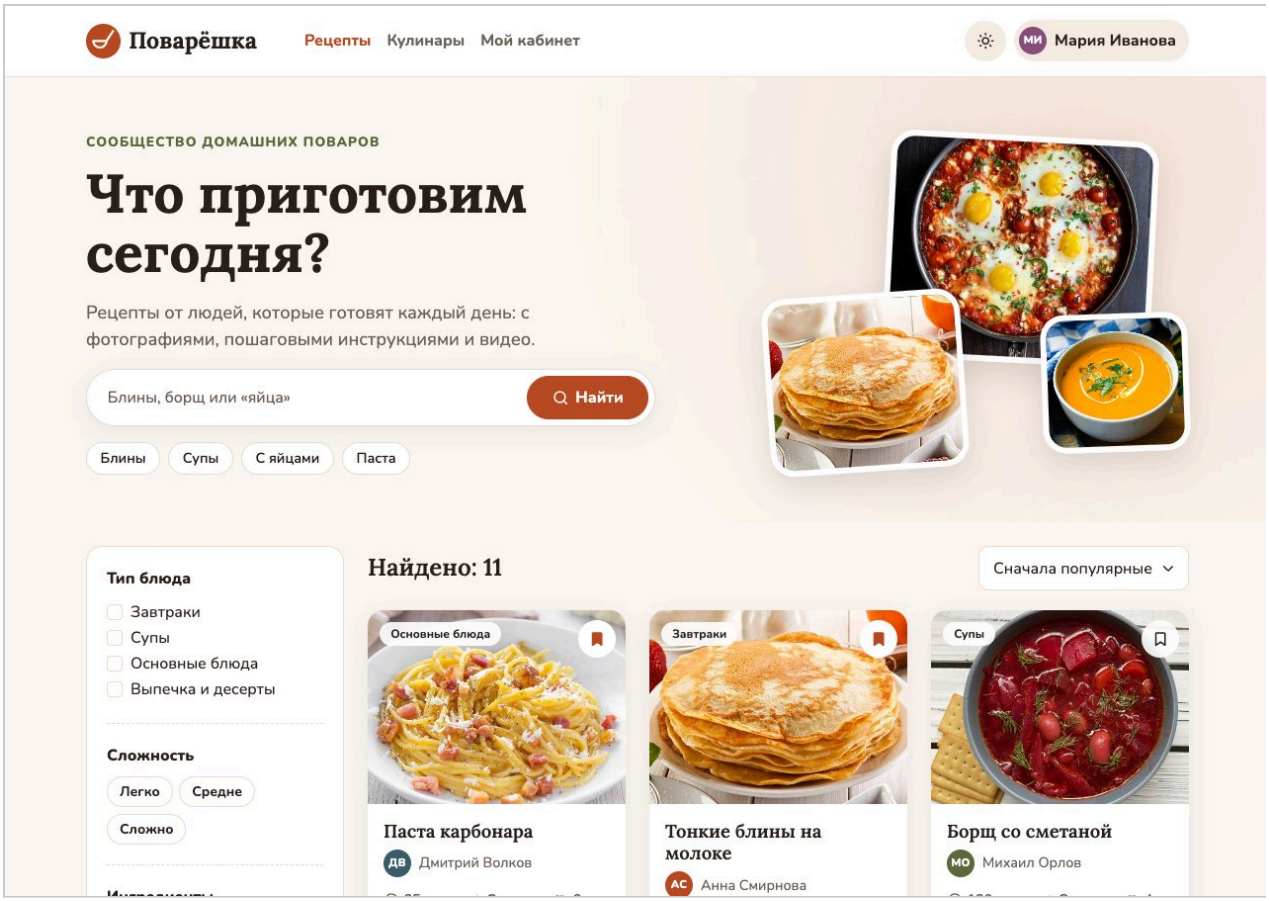


Рисунок 5. Главная: поиск рецептов (маршрут /)

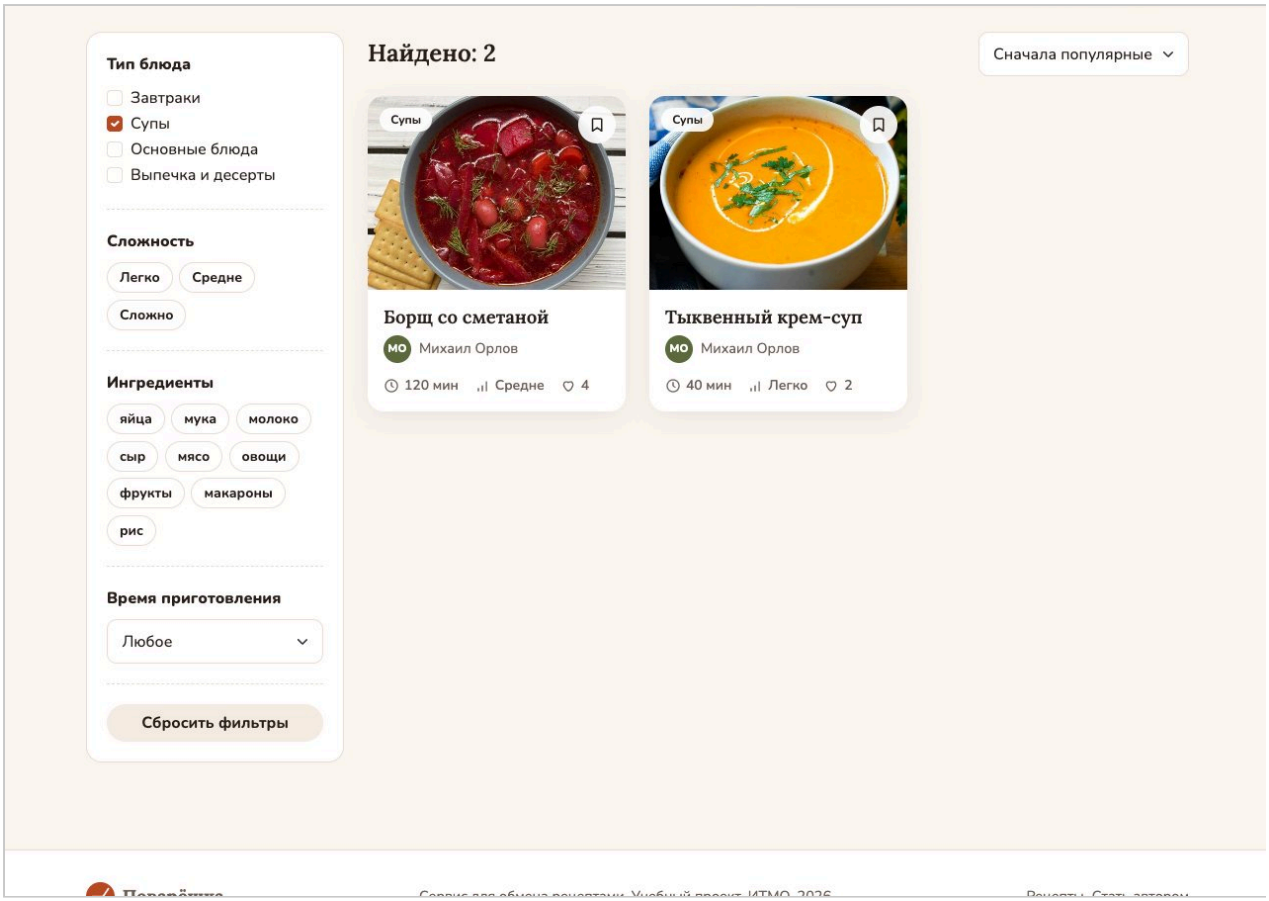


Рисунок 6. Фильтр из адреса /?type=Супы — так работает ссылка «Супы» в хлебных крошках

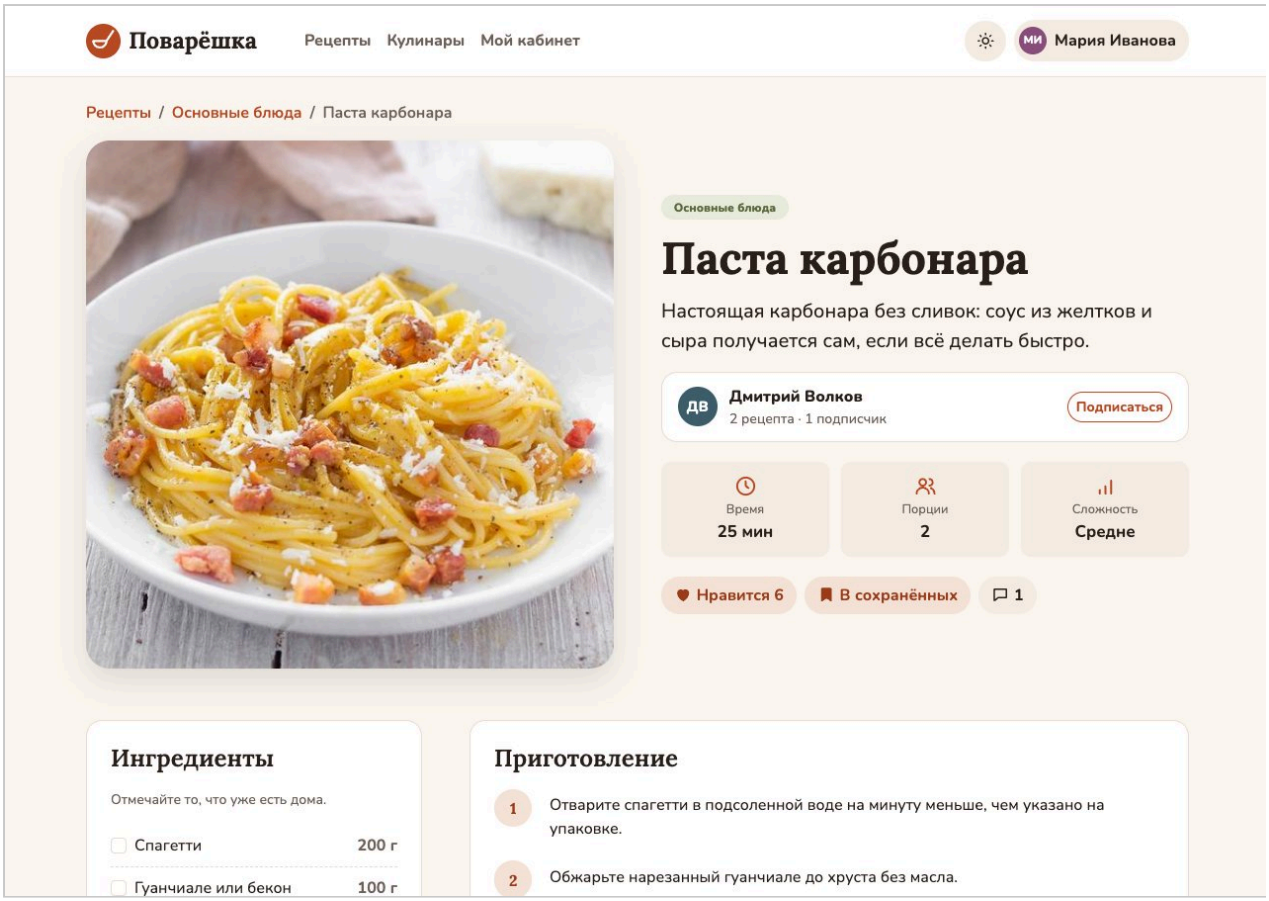


Рисунок 7. Страница рецепта /recipes/4

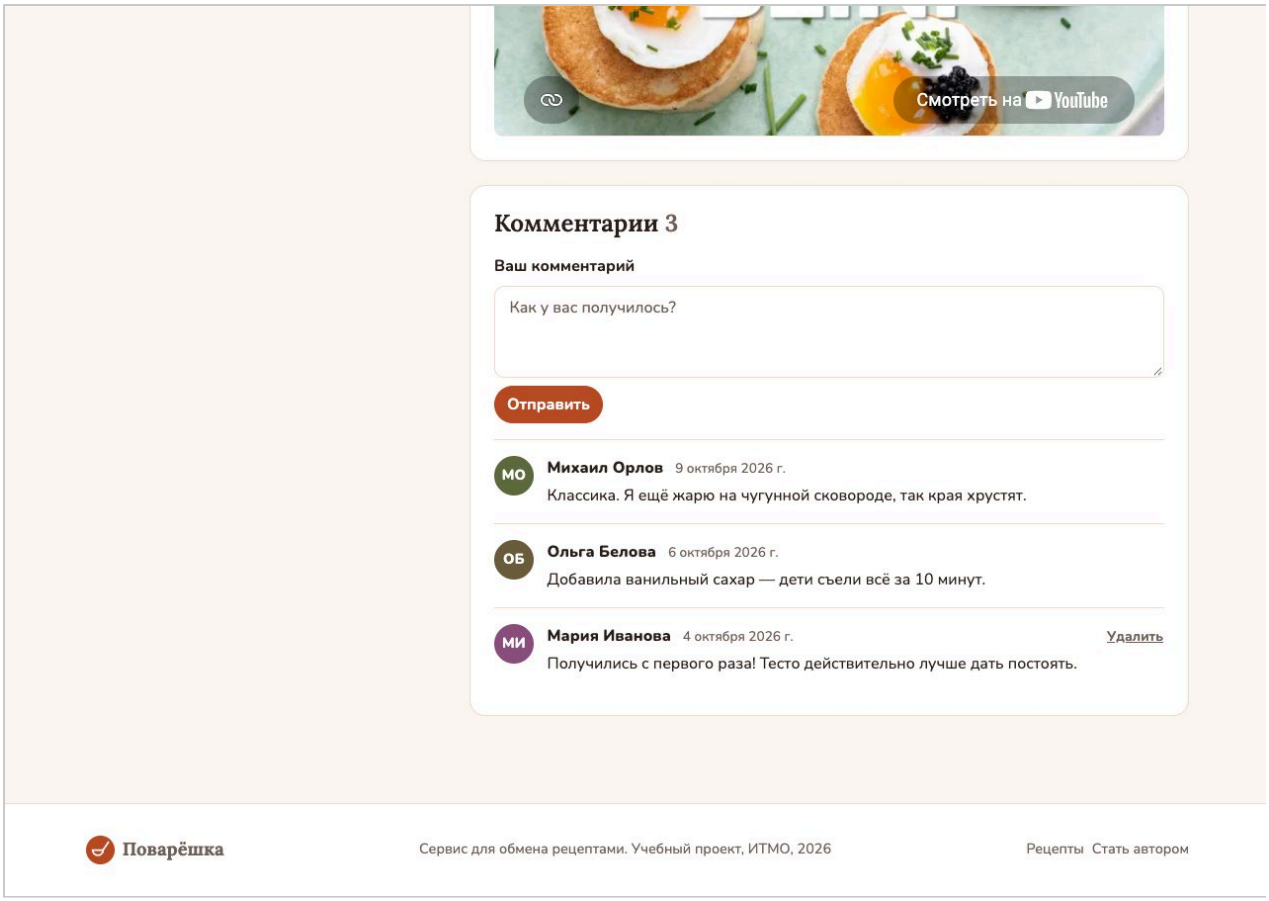


Рисунок 8. Комментарии (компонент CommentsSection)

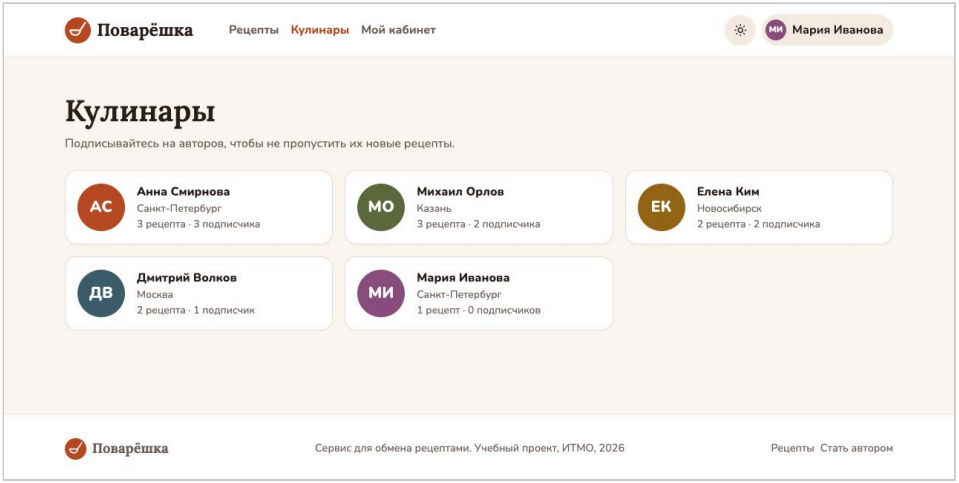


Рисунок 9. Кулинары /cooks

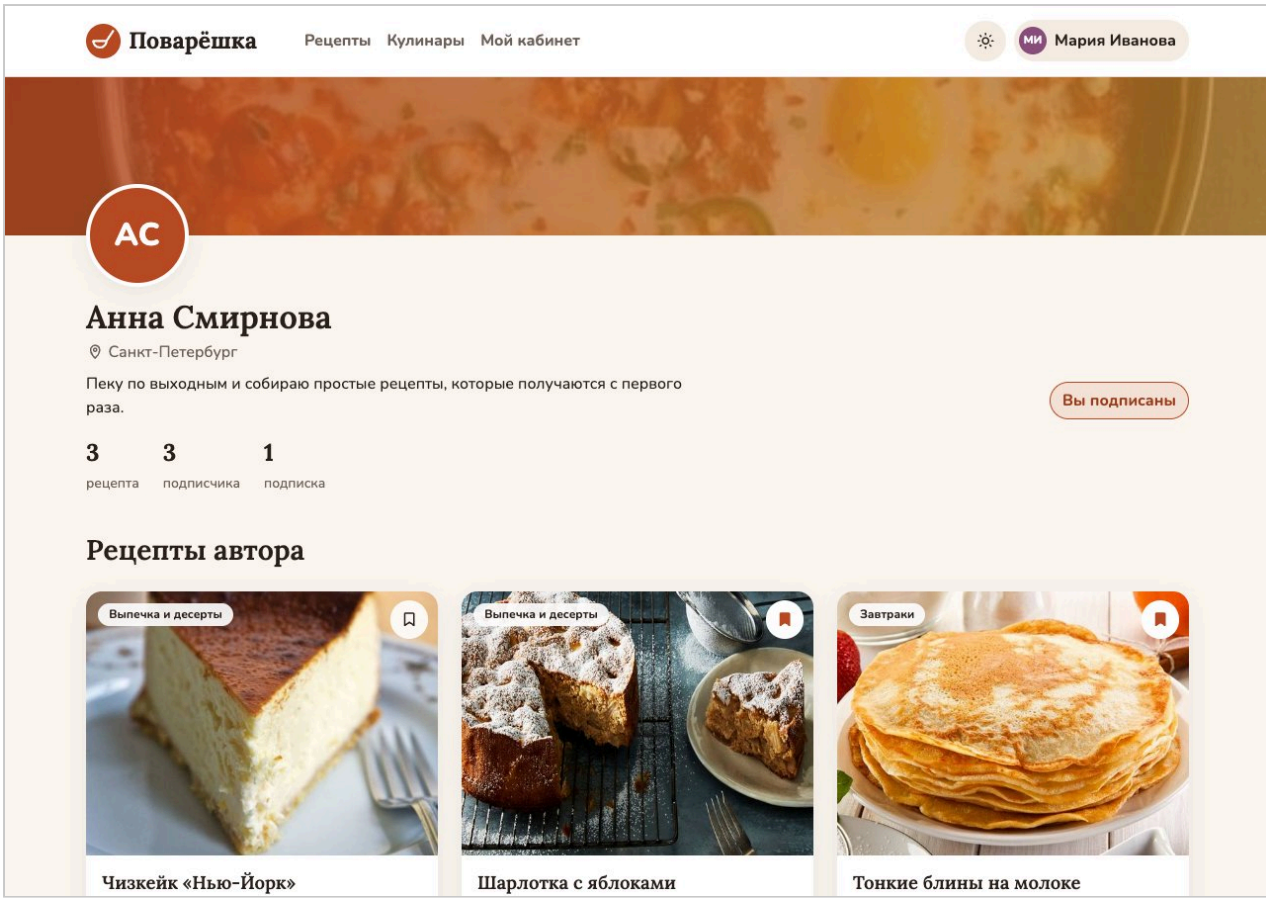


Рисунок 10. Страница кулинара /cooks/1 (ProfileHeader + RecipeGrid)

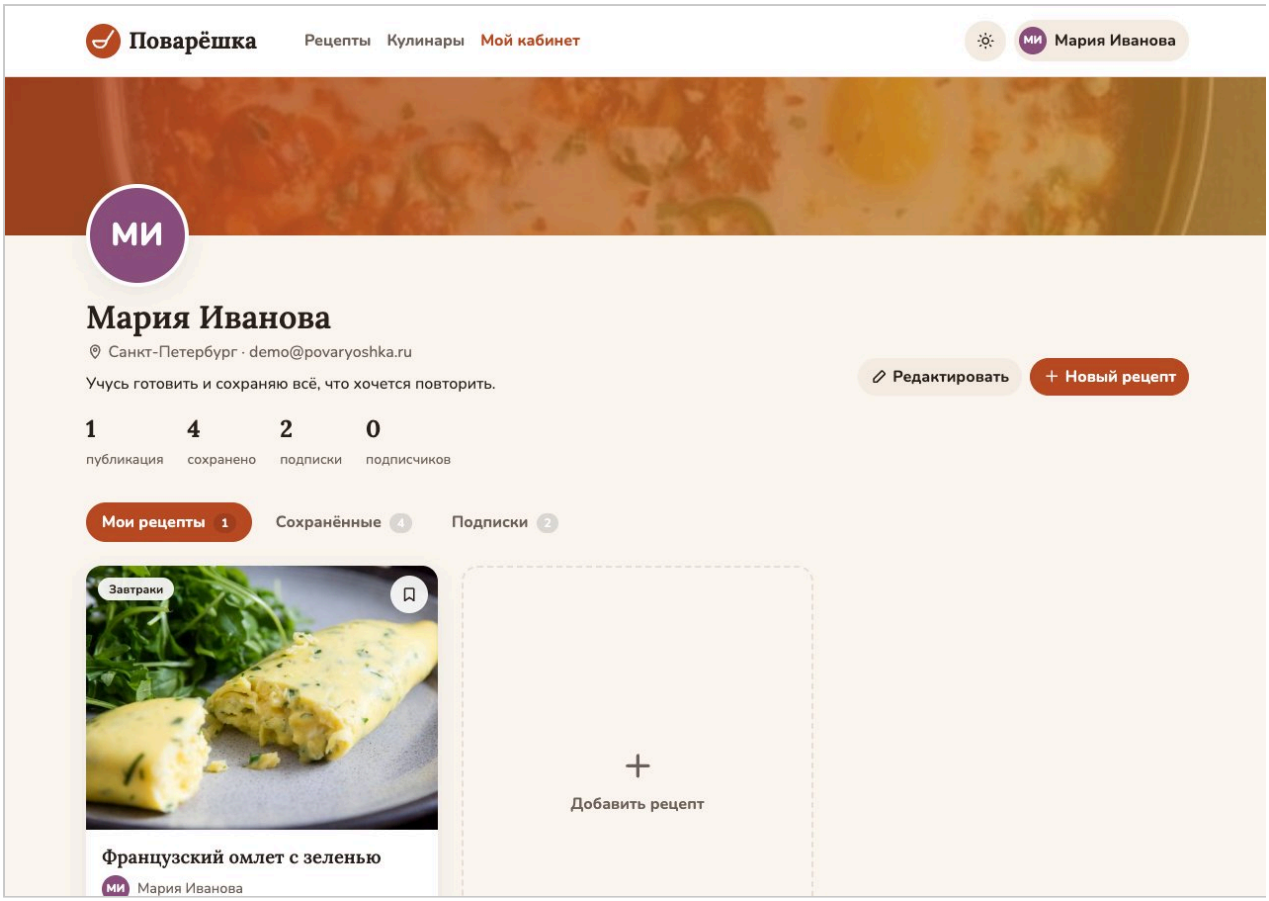


Рисунок 11. Личный кабинет /profile

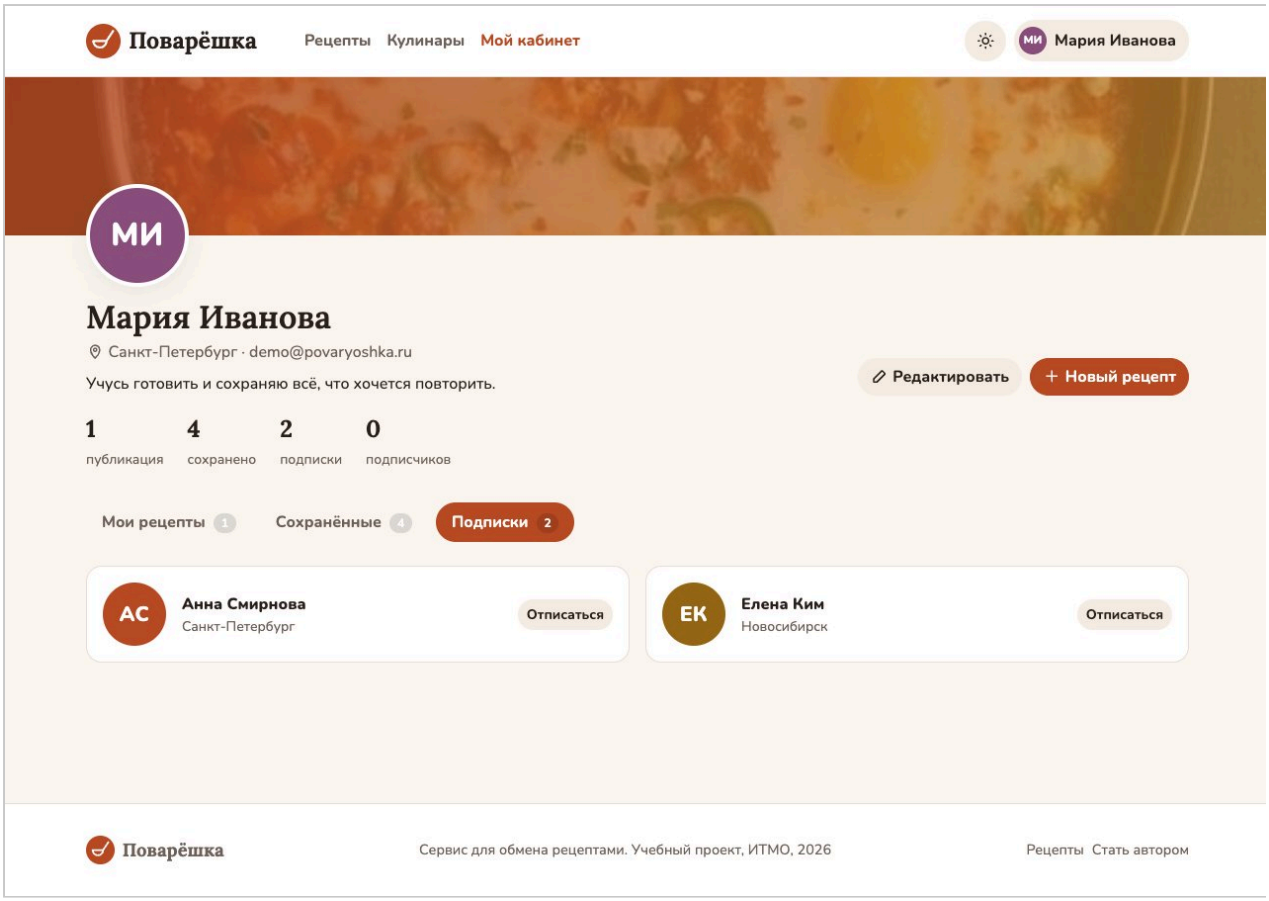


Рисунок 12. Вкладка «Подписки» (CookCard со слотом для кнопки)

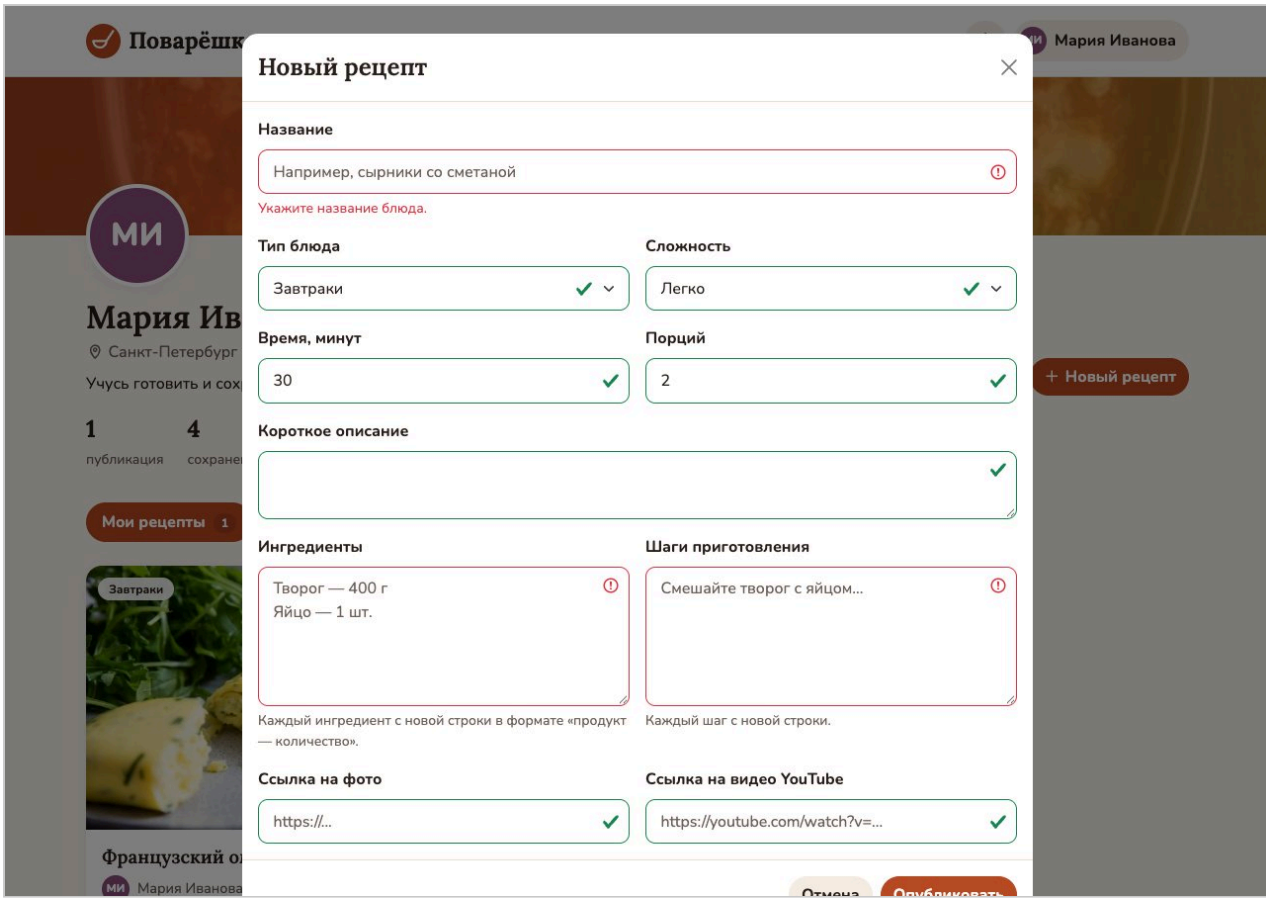


Рисунок 13. RecipeFormModal на базе BaseModal: проверка формы

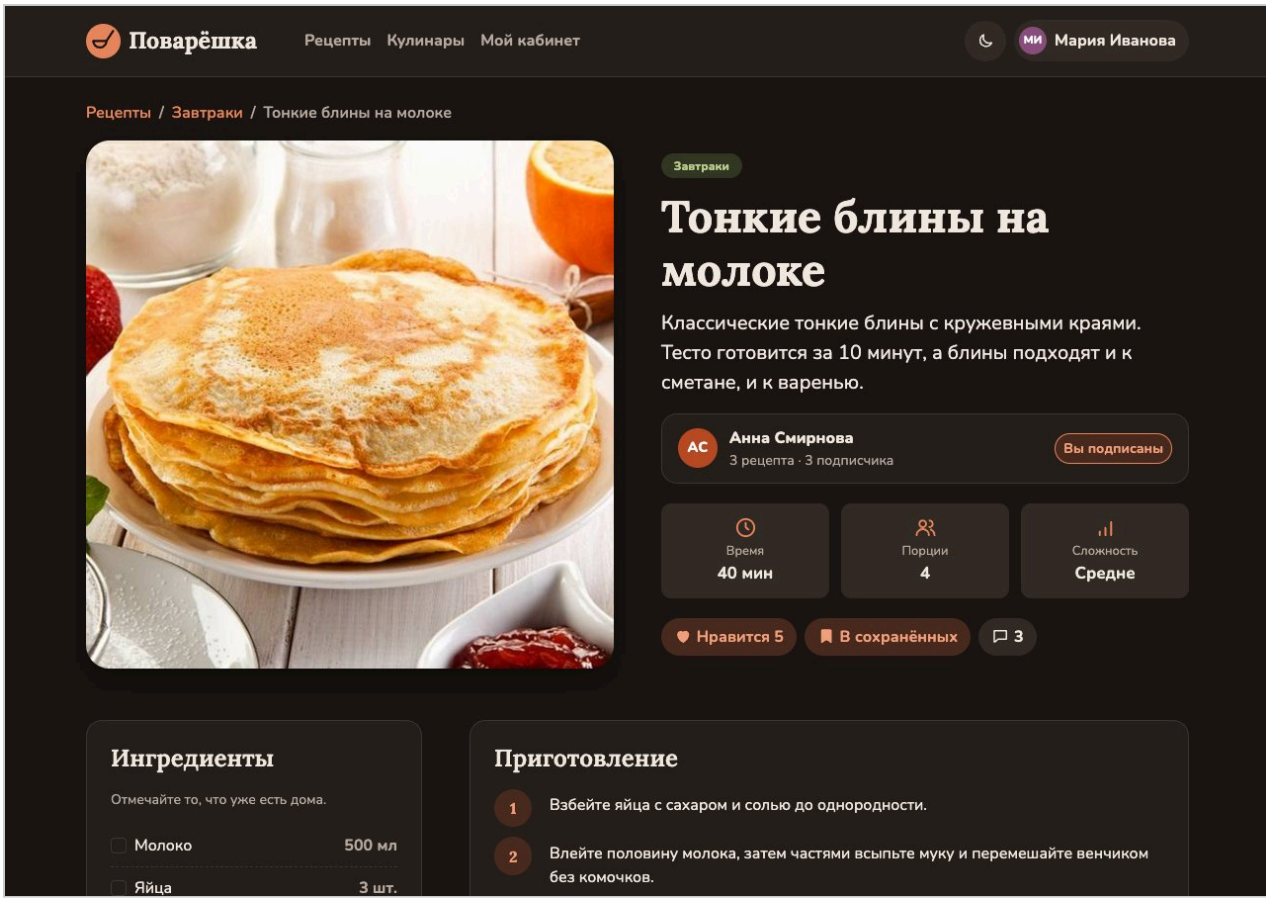


Рисунок 14. Тёмная тема (useTheme)

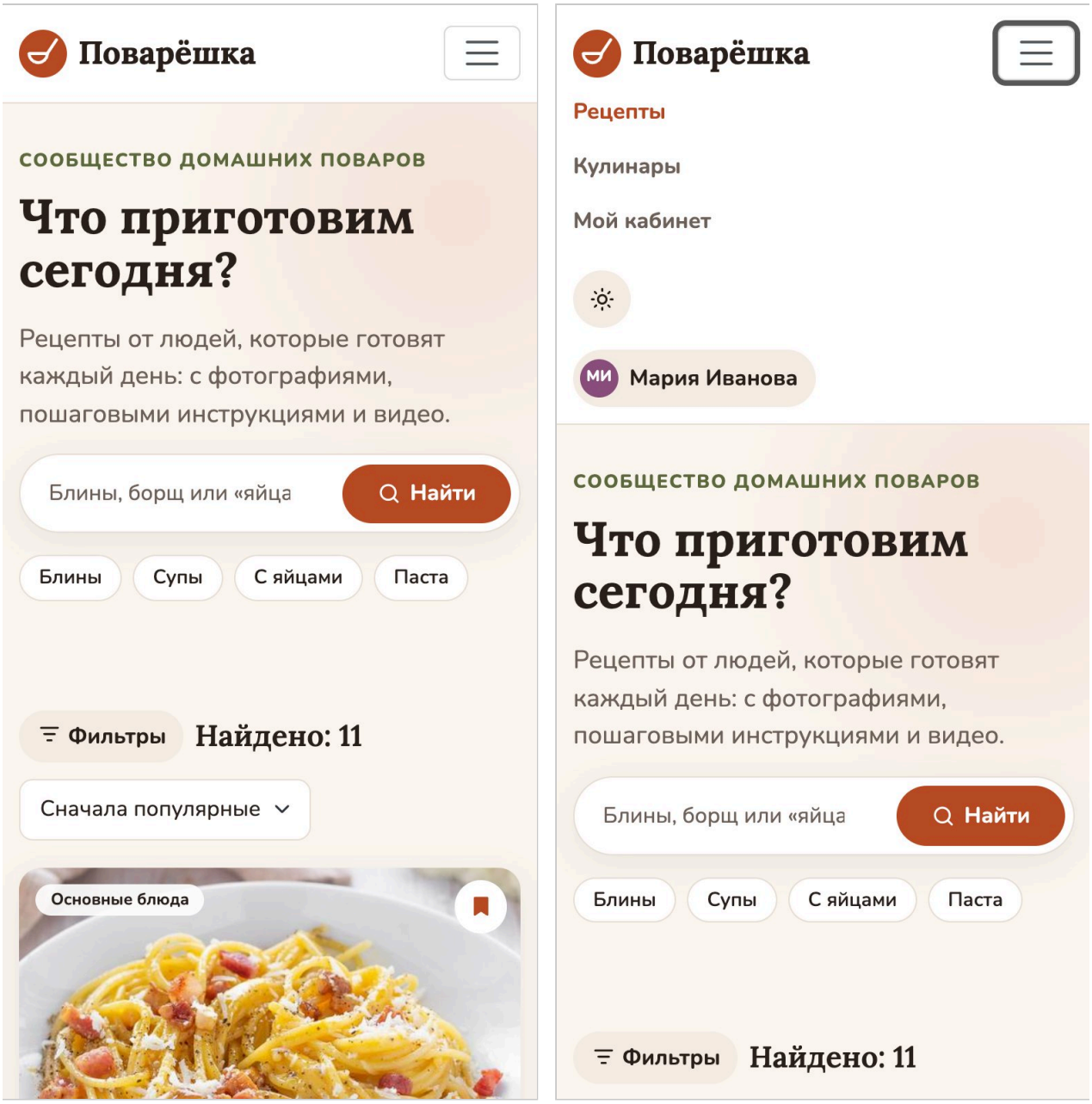


Рисунок 15. Мобильная версия и раскрытое меню

8. Проверка

- `npm run lint` (oxlint и ESLint с правилами для Vue) проходит без замечаний.
- `npm run build` собирает приложение. Каждая страница попадает в отдельный файл, например `RecipeView-….js` (9 КБ) или `ProfileView-….js` (15 КБ).
- Сквозной сценарий прогнан автоматически в браузере против запущенного API. Проверены:
  - фильтры и параметры из адреса;
  - guard кабинета и запрет входа для вошедшего пользователя;
  - переход на вход, если в `localStorage` лежит недействительный токен, а пользователь открыл кабинет;
  - вход с неверным и верным паролем, сохранение сессии в `localStorage`;

    - создание, изменение и удаление рецепта;
    - лайк, комментарий, подписка, сохранение;
    - переключение вкладок стрелками;
    - редактирование профиля, выход, регистрация, страница 404, кнопка «Назад» браузера;
    - перенос фокуса на `<main>` после перехода.
- Доступность, достигнутая в ДЗ2, сохранилась. axe-core (наборы WCAG 2.0/2.1 A и AA и best-practice) не нашёл нарушений ни на одном маршруте приложения ни в светлой, ни в тёмной теме.

Вывод

Сайт «Поварёшка» переведён на Vue 3 и стал одностраничным приложением. Все требования лабораторной выполнены:

- подключён Vue Router: восемь маршрутов с ленивой загрузкой, параметрами и navigation guards на `meta`;
- работа с API идёт через axios: общий экземпляр с перехватчиками и классы API с внедрением зависимости;
- интерфейс разделён на страницы, лейаут и 17 компонентов со свойствами, событиями, слотами и `v-model`;
- повторяющаяся логика (сохранения, подписки, обработка ошибок, уведомления, тема, поиск) вынесена в шесть composables.

Глобальное состояние авторизации хранится в Pinia с сохранением в `localStorage`. Доработки домашних работ перенесены в приложение: доступность, тёмная тема, SVG-спрайт.